PROJECT DOCUMENTATION · VERSION 1.0

CampusSafe

by **Lanchenba Mutum**

A hold-to-send safety loop for one real campus.

CampusSafe lets a student at Graphic Era (Deemed to be) University raise a **hold-to-send SOS** with live GPS, **report an incident** with a photo, or file an **anonymous report** tracked by a code. Campus command watches every case on a **live console**, assigns a **responder**, and each status change flows back to the student step by step.



3
roles: student, responder and admin, each with its own app

51
server functions behind one typed, validated API

20
Operating-System concepts implemented and mapped to code

9
SQL migrations: 19 tables, a view and a trigger

AUTHOR
Lanchenba Mutum

SOURCE CODE
github.com/1anc6/Campus-Safe

HOSTING
Vercel · Neon Postgres

STATE DOCUMENTED
20 October 2026

CAMPUS
Graphic Era University, Dehradun

CONTRIBUTORS
Monica Agum · Lydia1616

Contents

16 CHAPTERS

01 **Summary and key numbers**
What was built, in one page

02 **The problem, and what CampusSafe does**
Four ways to ask for help

03 **Timeline**
Eight days of commits, with teammates

04 **System architecture**
From a phone to Postgres and back

05 **The safety loop, feature by feature**
SOS, reports, anonymous, Safe Walk, broadcasts

06 **Operating-System concepts in the code**
Scheduling, aging, semaphores, races

07 **Database design**
Tables, constraints, indexes, view, trigger

08 **API: the server functions**
All 51, grouped by area

09 **Frontend and design system**
Soft UI, colour rules, screens

10 **Security, privacy and abuse protection**
Sessions, validation, rate limits, anonymity

11 **Testing and quality**
Unit tests, type checks, visual review

12 **Deployment and operations**
Vercel, Neon, migrations, cron

13 **Limitations**
Read before relying on it

14 **Complete inventory**
Every library, service and tool

15 **Future work**
What to build next, in order

16 **Glossary and credits**
Terms to know for a viva

HOW TO READ THIS DOCUMENT

Chapters 1–4 give the big picture. Chapters 5–8 explain the internals and are the ones to study for viva or interview questions; chapter 6 maps the Operating Systems syllabus and chapter 7 the DBMS syllabus onto real files. Chapter 13 is the honest assessment. Chapter 14 lists everything used, and chapter 16 defines the technical terms.

WHERE THE FACTS COME FROM

Every number here was counted from the repository on 20 October 2026: commits from `git log`, tables from `migrations/*.sql`, functions from `src/backend/functions/`, tests from the `*.test.*` files. Nothing is estimated. The project has **no measured response-time, uptime or usage figures**, so none appear here.

01 OVERVIEW

Summary and key numbers

CampusSafe is a full-stack web application that answers one question for a student in trouble on campus: *how do I reach someone who can help, right now, and know they are coming?* It runs in any phone browser, installs to the home screen, and is built around the real buildings and gates of Graphic Era University, Dehradun.

2 s
Hold on the SOS button to arm it, then a 5-second countdown that can still be cancelled

5
SOS steps the student sees: Sent → Received → Assigned → Arriving → Resolved

26
Real campus places with OpenStreetMap coordinates, from Gate 1 to the Dispensary

54
Git commits, 10–17 October 2026, including 11 merged teammate pull requests

~20,700
Lines of TypeScript, TSX, SQL, CSS and scripts in 120 source files

43
Automated unit tests across 6 files (scheduling, roles, migrations, scripts)

What was built

- **A student app** (phone-first, installable): hold-to-send SOS with live GPS, a live SOS screen with two-way chat, incident reports with a photo, anonymous reports, "Walk with me", campus broadcasts, a medical ID and a history of their own cases.
- **A responder field app** at `/staff`: assigned alerts, the student on a map, one-tap navigation and calling, and status updates.
- **An admin console** at `/admin`: a live dashboard, a priority queue that ages waiting cases, SOS map, incident and anonymous queues, broadcasts, user and responder management, analytics and a Ctrl/Cmd+K command palette.
- **A typed backend** of 51 server functions that check who is calling, what they may do and whether the input is valid before any SQL runs.
- **A PostgreSQL schema** built from 9 ordered migrations, with constraints, partial indexes, foreign keys, a trigger and a view.
- **Twenty Operating-System ideas** written into real features: priority scheduling with aging, a counting semaphore per responder, atomic compare-and-swap, a watchdog timer, exponential backoff and more.
- **A soft-UI design system** with equal light and dark themes, documented in `DESIGN.md`.

The guiding principle

> **SECONDS MATTER, AND EVERY REPORT LANDS SOMEWHERE**
>
> The path to help is never more than one deliberate action away, but it is never an accident either: the SOS needs a two-second hold and gives five seconds to cancel. Once sent, the student always sees what happened next. Red is reserved for SOS alone, so on any screen or map the one red thing is the person who needs help. Two more rules follow: **anonymity is real** (no account, a tracking code, nothing tying it back), and the app knows **one campus precisely**, with real places and routes rather than a generic map.

CONTEXT

02 The problem, and what CampusSafe does about it

The problem

A student who feels unsafe on campus today has three poor options: phone security and try to describe where they are, message a group chat and hope someone sees it, or say nothing. A phone call does not carry a location. A group chat has no owner, no priority and no record. And many students will not report harassment or ragging at all if it means giving their name.

Campus security has the opposite problem: reports arrive through many channels with no shared picture of what is open, what is urgent, and who is already handling it.

The approach: four ways in, one console out

CHANNEL	WHAT THE STUDENT DOES	WHAT COMMAND GETS
SOS	Holds the SOS button for 2 s; a 5 s countdown then sends it with the best GPS fix available	A red pin and live GPS trail, the student's medical ID, a chat thread, and a suggested responder
Incident	Picks a category, place, severity (P1–P4) and time, writes what happened, optionally adds a photo	A case in the priority queue with a 5-step status, assignment and a message thread
Anonymous	Files a report with no account, as victim, witness or someone who heard about it, and receives a tracking code	A case with no identity attached; replies go through the code at `/track/<code>`
Safe Walk	Says where they are going and picks 5, 10, 15, 20 or 30 minutes	Nothing, unless the student misses the check-in; then the walk escalates into an SOS automatically

Who uses it

ROLE	LANDS ON	WHAT THEY CAN DO
`student`	/	Hold SOS with live location, report incidents, report anonymously, start a Safe Walk, track their cases, keep a profile and medical ID. Students sign themselves up.
`responder`	/staff	See SOS alerts assigned to them, navigate, call the student, update status. Created by an admin.
`admin`	/admin	Live dashboard; SOS, incident and anonymous queues; broadcasts; users and responders; analytics.
anyone	/anonymous	File an anonymous report and follow it at `/track/<code>`. No account needed.

03 HISTORY
Timeline

CampusSafe was built between **10 and 17 October 2026**: 54 commits on `main` over seven working days, including eleven pull requests from two teammates. Hours were not tracked with a timer, so the table below shows only what the git history proves (times are India time).

DAY	COMMIT WINDOW	COMMITS	WHAT WAS BUILT
Fri 10 Oct	06:06–22:48	11	First full platform (student SOS, reports, responder app, admin console); folders split into frontend and backend; sign-in fixed for Vercel URLs; share preview; faster page switches. Evening: **fix for SOS never sending from real phones**, a liquid-glass redesign, then **Walk with me**, campus broadcasts and first-run onboarding.
Sat 11 Oct	00:18–15:20	3	Builder tooling removed; **OS scheduling features** (aging queue, responder semaphore, load balancing); login redesign; migration 0009 adds foreign keys, the `updated_at` trigger and the `open_cases` view.
Mon 13 Oct	13:18–22:01	17	Teammate pull requests #1–#4 (README, `package.json`, `migrate.mjs`); SOS location accuracy fix; admin activity feed; password-strength meter; warm charcoal dark mode; CSS cleanup.
Tue 14 Oct	19:43	1	Ctrl/Cmd+K command palette for the admin console.
Wed 15 Oct	21:13–21:42	14	Teammate pull requests #5–#11 (sign-out plan, profile, styles, `run.mjs`, `PRODUCT.md`, Prettier config) reviewed and merged.
Thu 16 Oct	17:56–23:24	6	**Soft-UI (neumorphism) redesign** in light and dark; **two-way SOS chat**; soft dropdowns; smaller home map on phones; Safe Walk guidance to the destination on the map.
Fri 17 Oct	00:41–00:49	2	Interaction motion; fixes for the clipped SOS counter and oversized map on phone admin screens.

A useful case study: "SOS never sends from real phones"

The first version worked on a laptop and failed on phones in the field. The fix (commit `abeb37d`, 10 Oct) and the follow-up accuracy fix (`2ce97f1`, 13 Oct) shaped the final SOS design:

1. **GPS only works on HTTPS.** Browsers refuse location on plain HTTP, so phones must be tested through the deployed site, not a LAN address.
2. **The first GPS fix is often poor.** After the countdown the app waits a short time for a more accurate fix, then sends anyway with the best one it has, marking the alert approximate if needed (`is_approximate`, `accuracy_m`).
3. **Mobile networks drop requests.** A failed SOS is now stored on the phone and retried with exponential backoff (4 s, 8 s, 15 s, then every 30 s) instead of failing silently.
4. **Students can be off campus.** A haversine distance check warns when a location is far from campus (beyond 2 km) so command can tell a real alert from a bad fix.

> **THE LESSON, IN ONE LINE**
>
> Test the emergency path on a real phone, on a real network, on the deployed URL, before trusting it.

04 BIG PICTURE

System architecture

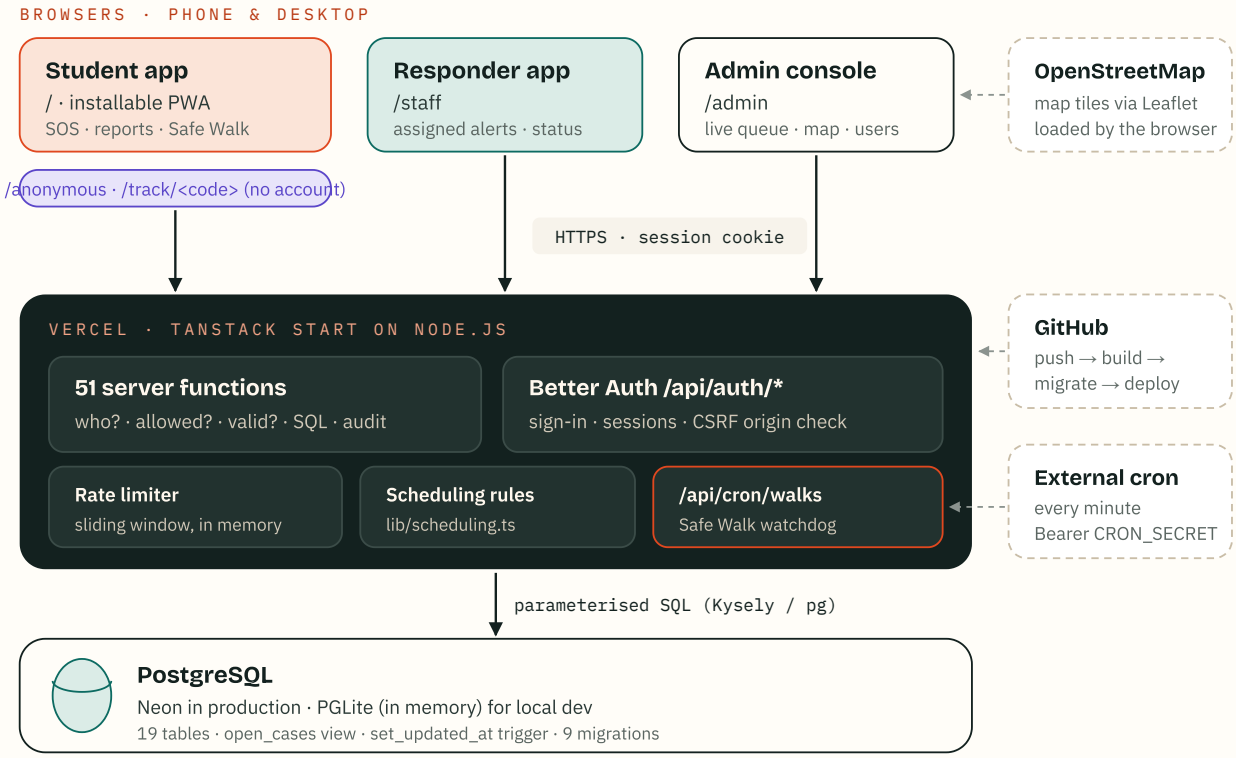


FIG 1 Production architecture. Solid arrows carry user requests; dashed grey links are browser-side map tiles, deployment and the scheduled watchdog.

How the pieces fit

TanStack Start lets a page call a backend function **as if it were a normal function**. At build time the call becomes an HTTPS request to the server, so the browser never sees backend code and never touches the database. Sign-in is a separate Better Auth handler mounted at `/api/auth/*`, storing users and sessions in the **same** Postgres database as everything else.

The code is split by where it runs: `src/routes` and `src/components` are what the user sees; `src/backend` is what the server does; `src/lib` is shared by both (types, campus data, scheduling rules); `migrations` is what the database looks like.

Every backend function follows the same five steps

1. **Who is calling?** `authMiddleware` reads the session cookie.
2. **Are they allowed?** Role and ownership checks: `requireAdmin`, "is this your SOS?".
3. **Is the input valid?** A `zod` schema rejects bad data, such as a latitude of 999.
4. **Do the work** with parameterised SQL, which is safe from SQL injection.
5. **Record it.** Important actions are written to `audit_log`.

Key design decisions

DECISION	WHY
Server functions instead of a hand-written REST API	One typed call from page to server; input schemas and return types are shared, so the two sides cannot drift apart.
Auth tables in the app's own Postgres	One database to back up and migrate; the session cookie stays on the app's own origin.
PGLite when no `DATABASE_URL` is set	Real Postgres compiled to WebAssembly, so `npm run dev:local` works with zero setup. Data resets on restart.
Polling plus tab-visibility refresh	Works on serverless hosting with no socket server; consoles refresh the moment a tab becomes visible again.
Scheduling rules as pure functions	`lib/scheduling.ts` has no I/O, so it runs the same on server and browser and is unit-tested.
Database constraints as the last line of defence	A unique partial index stops duplicate open SOS alerts even if two requests race past the application check.

05 INTERNALS

The safety loop, feature by feature

5.1 SOS: from a held button to a resolved alert

1 **Sent** — Student holds 2 s, countdown ends, alert is created

2 **Received** — Campus command has seen it

3 **Assigned** — A responder unit takes a semaphore slot

4 **Arriving** — Responder is on the way

5 **Resolved** — Slot released; student marked safe

The life of one SOS

1. **Arm.** The SOS button (`sos-button.tsx`) runs a real 2,000 ms timer; letting go early cancels it. A completed hold opens a 5-second countdown that can still be cancelled.
2. **Locate.** The flow (`sos-flow.tsx`) asks for the phone's position and briefly waits for an accurate fix before sending with the best one available. A screen Wake Lock asks the phone not to sleep while sending.
3. **Create.** `createSos` checks the session, then inserts into `sos_alerts`, `sos_locations`, `threads` and `audit_log`. A unique partial index guarantees one open SOS per student: if two presses race, the second insert is rejected and the first alert is returned.
4. **Track.** The student lands on `/sos/<id>`. Every few seconds `pingSosLocation` adds a GPS point; pings less than 2 seconds apart are merged.
5. **Dispatch.** The admin console shows a red pin, the trail and the student's medical ID. `pickResponder` suggests the least-busy unit; `assignSos` acquires its slot atomically.
6. **Talk.** Student and command chat on the alert's thread, with quick replies such as "Someone is following me".
7. **Close.** The student taps **I'm safe**, or command resolves it. `closeSos` uses a compare-and-swap so the responder's slot is released exactly once.

> **IF THE NETWORK FAILS**
>
> The alert is kept on the phone and retried with exponential backoff: 4 s, 8 s, 15 s, then every 30 s. The student sees a clear retry state instead of a silent failure.

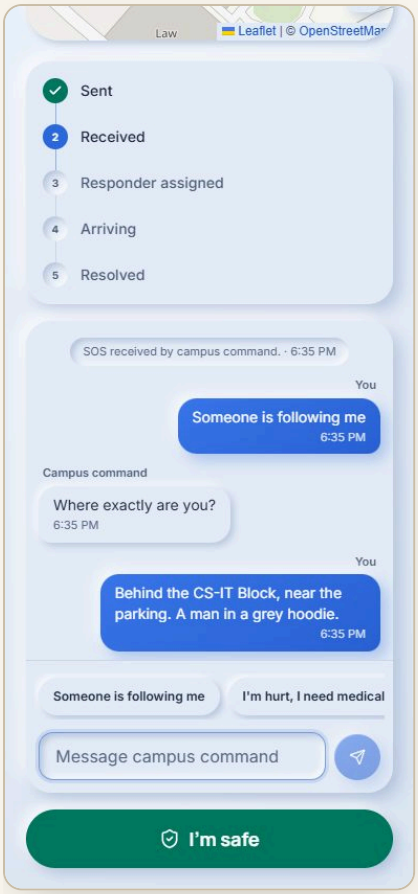


FIG 2 Live SOS screen: the status stepper, two-way chat with quick replies, and the green "I'm safe" button.

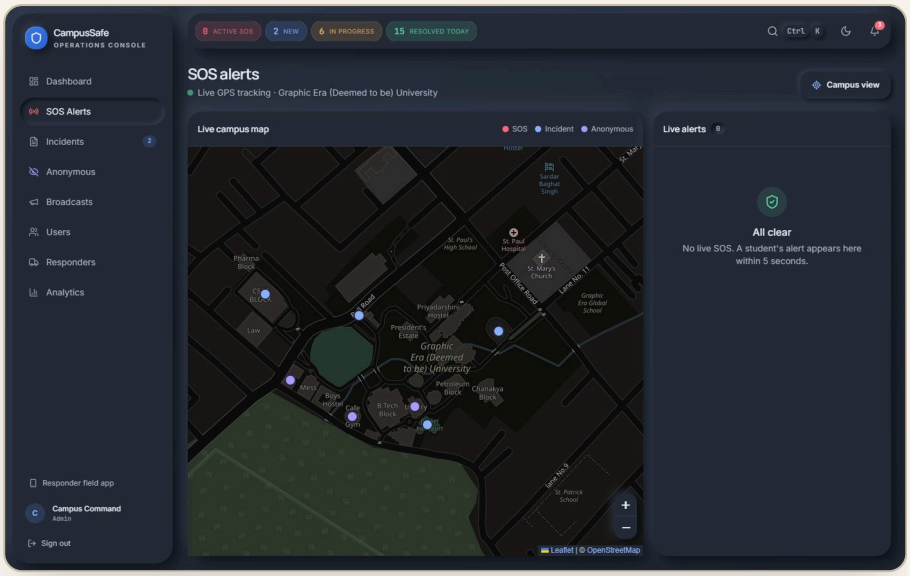


FIG 3 Admin SOS map in the dark theme. SOS pins are red, incidents blue, anonymous reports violet, on Graphic Era's real streets and buildings.

5.2 Incident reports

A signed-in student picks one of **9 categories** (harassment, theft, medical, fire, infrastructure, ragging, suspicious, accident, other), one of the 26 campus places, a severity and when it happened, then describes it. A photo is optional: the form downsizes it in the browser, and the server accepts only PNG, JPEG or WebP data under about 3 MB. Case lists never send photos; only the case view loads one.

SEVERITY	LABEL	MEANING
P1	Critical	Immediate danger
P2	High	Urgent, not life-threatening
P3	Medium	Needs attention today
P4	Low	Non-urgent record

Cases move through **Reported** → **Under Review** → **In Progress** → **Resolved** → **Closed**, and every change appears on the student's timeline.

5.3 Anonymous reports

Anyone can open `/anonymous` without an account and report as a **victim**, a **witness** or someone who **heard** about it. The reply is a tracking code such as `CS-P2LW-55CK`: eight characters drawn with cryptographic randomness from a 31-

character alphabet that leaves out look-alikes (0/O, 1/I/L), using rejection sampling so every character is equally likely. That is about 8.5×10^{11} possible codes, and guessing is rate-limited.

> **WHAT IS NEVER STORED**
>
> No user id, no email and no IP address. Rate limiting uses a SHA-256 hash of the IP with a random per-process salt, held only in memory. The admin console shows "Reporter is anonymous. Replies route through CS-..." and the timeline records "Identity was never stored".

5.4 Walk with me (Safe Walk)

A student walking alone types a destination and picks 5, 10, 15, 20 or 30 minutes. The map guides them to the destination. If they do not check in by the deadline, the walk **escalates into an SOS** on its own. This is a watchdog timer: an external scheduler calls `/api/cron/walks` every minute with a secret, so overdue walks escalate even when nobody has the app open. A compare-and-swap (`update … where status = 'active'`) makes sure a walk escalates only once.

5.5 Campus broadcasts, profile and medical ID

- **Broadcasts.** Admins send campus-wide alerts at *info*, *warning* or *critical* level; students acknowledge them (`campus_alerts` , `alert_acks`).
- **Medical ID.** Blood group (checked against the 8 valid groups), course and an emergency contact. Responders see it during an SOS so they can help faster.
- **Call Campus Security** is one tap from the home screen. The numbers in `src/lib/campus.ts` are still placeholders and are labelled as such in the app.

06 OPERATING SYSTEMS SYLLABUS

Operating-System concepts in the code

The scheduling rules live in one small, unit-tested file, `src/lib/scheduling.ts`. The rest are spread across the features they protect. Every row below points at real code.

6.1 Priority scheduling with aging: a worked example

The live queue is sorted by three keys, like a multilevel scheduler: **SOS first**; then by **aged severity**; then **first come, first served**. Every 30 minutes of waiting lifts a case one level (`AGING_STEP_MIN = 30`), so a P4 is treated like a P1 after 90 minutes and can never starve behind a stream of new P1s.

CASE AT 10:00	FILED	WAITED	SEVERITY	AGED TO	QUEUE POSITION
SOS · Boys Hostel	10:00	0 min	—	—	**1** · SOS always first
Anonymous · Central Library	09:00	60 min	P4	P2	**2** · ties on P2, filed earlier
Incident · Parking	09:50	10 min	P2	P2	**3**
Incident · Mess	09:55	5 min	P3	P3	**4**

6.2 A counting semaphore per responder

Each unit may hold at most **2 live SOS alerts** (`MAX_ACTIVE_SOS_PER_RESPONDER`). Assigning is **P()**: an atomic "add 1 only if below 2" on `responders.active_sos`. Resolving is **V()**: subtract 1. A `CHECK (active_sos >= 0)` constraint means the count can never go negative. `pickResponder` skips full and off-duty units, prefers the least loaded, and breaks ties round-robin by who has waited longest since their last alert.

UNIT	STATUS	LIVE SOS	LAST ASSIGNED	RESULT
Campus Security A	available	1 / 2	09:40	ready
Night Patrol	on duty	1 / 2	09:10	**suggested**: same load, waited longer
Medical Unit 1	on duty	2 / 2	09:55	skipped: semaphore full
Fire & Rescue	off duty	0 / 2	—	skipped: off duty

6.3 All twenty concepts

OS CONCEPT	WHERE	HOW
Priority scheduling	`compareQueueItems`	SOS first, then severity, ties first-come-first-served
Aging (no starvation)	`agedLevel`	+1 level per 30 min waiting; the queue shows "aged P4→P2"
Load balancing + round-robin	`pickResponder`	Least-busy unit; ties go to the longest-waiting unit
Counting semaphore	`acquireResponderSlot` / `releaseResponderSlot`	At most 2 live SOS per unit; P() on assign, V() on resolve

OS CONCEPT	WHERE	HOW
Race condition, fixed atomically	`openSosForUser` + `sos_alerts_one_open_idx`	Two simultaneous presses cannot create two alerts
Atomic compare-and-swap	`escalate`, `closeSos`	`update … where status = 'active'`: only the first racer wins
Optimistic concurrency	`assignSos`	Re-assignment applies only if nobody changed the alert since it was read
Mutual exclusion	`backend/db.ts`	Migrations run one at a time on a single promise chain, each in a transaction
Watchdog timer	`escalateOverdueWalks`	A missed Safe Walk check-in becomes an SOS
Cron / background daemon	`routes/api/cron/walks.ts`	An external scheduler runs the watchdog every minute
Shared memory / caching	admin counters (`admin.ts`)	All open consoles share one result for 3 seconds
Resource limits / throttling	`core/rate-limit.ts`, GPS pings	Sliding-window limits; pings < 2 s apart are merged
Timers and interrupts	`sos-button.tsx`	2-second hold on a real timer; releasing early cancels it
Buffering / spooling	`sos-flow.tsx`	Offline SOS queued on the phone, retried with exponential backoff
Event-driven I/O	`admin-shell.tsx`	Refresh when the tab becomes visible instead of constant polling
Volatile vs persistent storage	PGLite vs Neon	Why local data resets but production data does not
Protection / access control	roles + server checks	Like OS permission levels; passwords hashed
Process creation and signals	`scripts/run.mjs`	Starts the dev server as a child process, forwards SIGINT/SIGTERM and its exit code
Atomic file write	`scripts/write-atomic.mjs`	Write a temp file, then `rename()`, so nobody sees a half-written file
Power management	Wake Lock during SOS	Asks the phone not to sleep while sending

Algorithms elsewhere: **haversine** great-circle distance, a circular **geofence** (on campus within 600 m of the centre, "near" within 2 km) and a **nearest-neighbour** search for the closest named place, all in `lib/campus.ts`.

07 DBMS SYLLABUS

Database design

PostgreSQL holds everything: sign-in data, profiles, cases, live GPS trails, messages and the audit trail. The schema is built by 9 SQL files applied in order and recorded in `_migrations`; old files are never edited.

Tables

TABLE	HOLDS
`user`, `session`, `account`, `verification`	Sign-in data managed by Better Auth; passwords are stored hashed, never in plain text
`profiles`	Name, role (`student` / `responder` / `admin`), phone, course, blood group, emergency contact, notifications-seen time
`sos_alerts`	One row per SOS: status, position and accuracy, assigned unit, a timestamp for every step
`sos_locations`	The live GPS trail, many points per alert; foreign key with `ON DELETE CASCADE`
`incidents`	Reports by signed-in students: category, place, severity, status, optional photo
`anonymous_reports`	Reports with no identity, only a `UNIQUE` random tracking code
`threads`	Messages on any case: student, command, responder or system
`responders`	Responder units, duty status, and the `active_sos` semaphore count
`safe_walks`	"Walk with me" sessions with destination and due time
`campus_alerts`, `alert_acks`	Campus-wide broadcasts and who has acknowledged them
`audit_log`	Who did what, when: the evidence trail
`campus_locations`, `incident_categories`	Reference data: the 26 places and 9 categories
`app_meta`, `_migrations`	Settings, and which migration files are already applied

Migrations

FILE	WHAT IT ADDS
`0001_auth.sql`	Better Auth tables: users, sessions, accounts, verification
`0002_schema.sql`	Profiles, SOS, GPS trail, incidents, anonymous reports, threads, audit log
`0003_student_role.sql`	Renames the `citizen` role to `student` and rewrites the `CHECK`
`0004_graphic_era_campus.sql`	Replaces placeholder places with Graphic Era's real buildings and gates
`0005_student_profile.sql`	Course, blood group (with a `CHECK`), emergency contact
`0006_notifications_and_indexes.sql`	Notification-bell state and partial indexes on open cases
`0007_safe_walks_and_campus_alerts.sql`	Safe Walk and broadcasts, with "one active walk per student"

FILE	WHAT IT ADDS
`0008_sos_scheduling.sql`	One open SOS per student; the responder semaphore column
`0009_view_trigger_foreign_keys.sql`	Foreign keys, the `updated_at` trigger and the `open_cases` view

DBMS concepts used

Constraints

Primary keys; `CHECK` on every status, severity, role and blood group; `UNIQUE` tracking codes; `active_sos >= 0`.

Partial indexes

Indexes that cover only open cases (`where status not in ('resolved','closed')`), so the live queue stays fast as history grows. A **unique** partial index enforces one open SOS per student.

Foreign keys, added safely

Added `NOT VALID` first, so every new row is checked at once, then validated; if old rows don't match, the key simply stays `NOT VALID` instead of blocking a deploy.

Trigger

`set_updated_at()` runs `BEFORE UPDATE` on incidents and anonymous reports, so `updated_at` is always correct whatever code made the change.

View

`open_cases` joins open incidents and open anonymous reports with `UNION ALL` into one shape; the notification bell reads it like a table.

Queries and transactions

Joins, aggregates, correlated sub-queries, `generate_series` for charts, and `row_number() over (partition by …)` for the latest points of each map trail. Each migration file runs in one transaction.

NEON VS PGLITE

With `DATABASE_URL` set, `src/backend/db.ts` connects to Neon, a persistent serverless Postgres. Without it, it starts PGLite: real Postgres compiled to WebAssembly, running in memory, so the app works with zero setup and the data resets on restart.

08 REFERENCE

API: the server functions

All 51 functions live in `src/backend/functions/`. Each is a `createServerFn` with an input schema; the role column shows who may call it.

FILE	COUNT	FUNCTIONS	CALLED BY
sos.ts	10	`createSos` · `pingSosLocation` · `markSosSafe` · `cancelSos` · `getSos` · `replySos` · `assignSos` · `setSosStatus` · `listMySos` · `listAssignedSos`	student, admin, responder
reports.ts	10	`createIncident` · `listMyIncidents` · `getIncident` · `assignIncident` · `createAnonymous` · `trackAnonymous` · `replyAnonymousPublic` · `staffSetIncidentStatus` · `staffSetAnonymousStatus` · `postThread`	student, anyone (anonymous), staff
admin.ts	9	`getAdminCounters` · `getAdminNotifications` · `markAdminNotificationsSeen` · `getAnalytics` · `getLive` · `listIncidentsAdmin` · `listAnonymousAdmin` · `listResponders` · `getActivity`	admin
profile.ts	9	`getMyProfile` · `updateMyProfile` · `updateMyPhone` · `changeMyEmail` · `listMyActivity` · `listPeople` · `setPersonRole` · `createAccount` · `resetPassword`	any signed-in user; admin for people
walks.ts	6	`getMyWalk` · `startWalk` · `extendWalk` · `endWalk` · `pingWalk` · `escalateMyWalk`	student
alerts.ts	5	`createCampusAlert` · `endCampusAlert` · `listCampusAlertsAdmin` · `getMyCampusAlerts` · `ackCampusAlert`	admin; student
viewer.ts	2	`getViewer` · `prepareSignIn`	everyone

HTTP endpoints outside the server functions

METHOD	PATH	PURPOSE
GET, POST	/api/auth/*	Better Auth: sign-up, sign-in, sign-out, session
GET	/api/cron/walks	Safe Walk watchdog; requires `Authorization: Bearer <CRON_SECRET>`, disabled when the secret is unset

Pages

URL	PAGE
/	Student home: campus map, SOS, Safe Walk, medical ID, my reports
/report · /reports · /reports/<id>	File an incident, list and follow your own cases
/sos/<id>	The student's live SOS screen

URL	PAGE
`/anonymous · /track/<code>`	Anonymous report and its tracking page (no account)
`/staff · /staff/<id>`	Responder field app: assigned alerts, one alert with map and status
`/admin`	Command dashboard with the live queue
`/admin/sos · /admin/incidents · /admin/anonymous`	Case queues and full case views
`/admin/alerts · /admin/users · /admin/responders · /admin/analytics`	Broadcasts, accounts, responder units, charts
`/login · /signup · /forbidden`	Sign-in, student self-registration, and the "not allowed" page

INTERFACE

09 Frontend and design system

The interface is built with React 19, TanStack Router (the file path is the URL) and Tailwind CSS 4. Maps use Leaflet over OpenStreetMap with Graphic Era's real building coordinates.

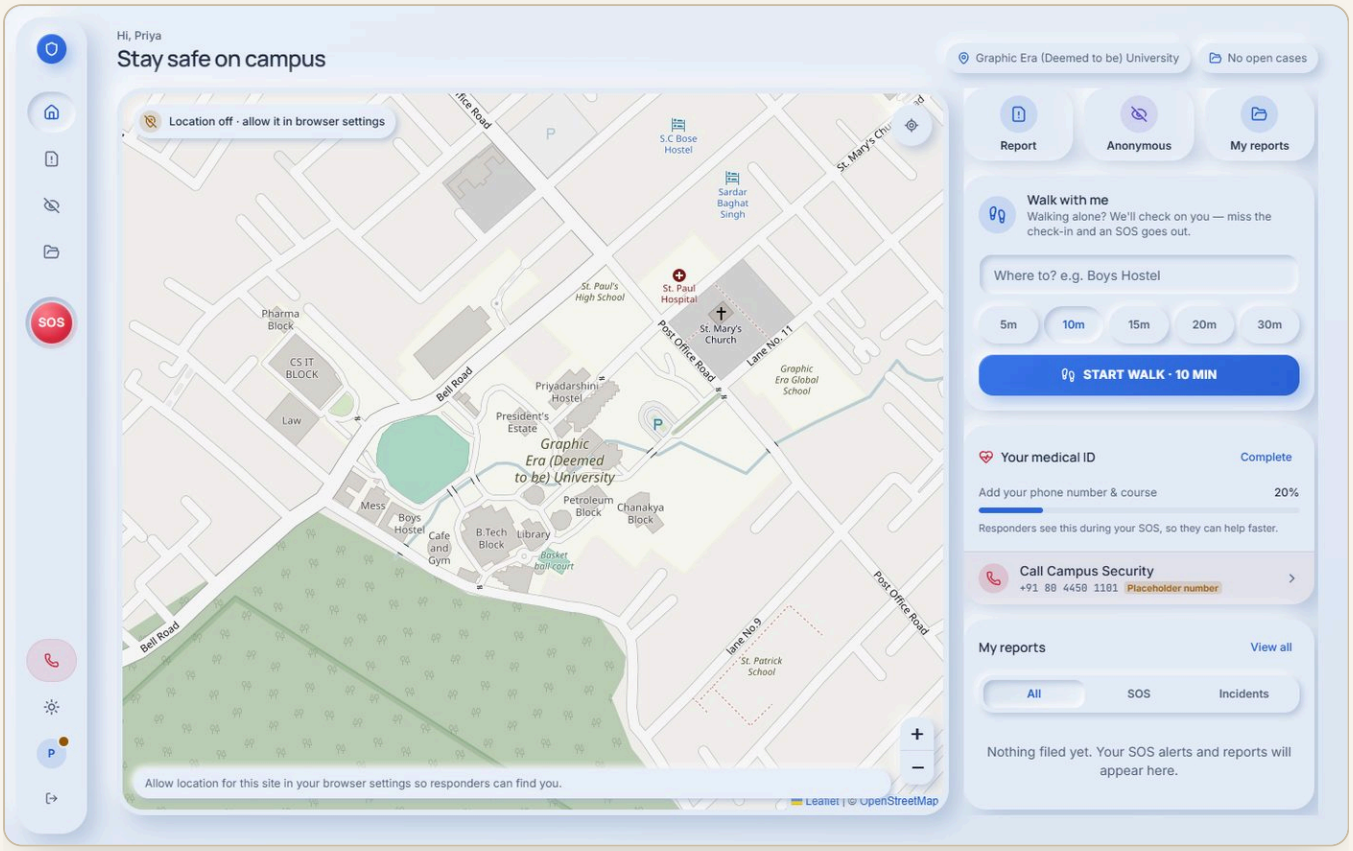


FIG 4 Student home on desktop: the campus map, the SOS button in the side rail, Report / Anonymous / My reports, Walk with me, the medical ID and the one-tap Campus Security call.

Soft UI, in two equal themes

Every panel and control is the same colour as the page and is shaped only by a soft light-and-shadow pair (neumorphism). **Raised** means you can press it; **pressed in** means it is selected or you are typing into it. Light and dark are equal peers; the moon/sun button switches them and the choice is remembered on the device.

COLOUR	TOKEN	RULE
SOS red	#c42239	**Means SOS only.** On maps, only an SOS pin is ever red.
Action blue	#2d6ade	Buttons and selected states.
Violet	#5b46c9	Marks anonymous flows everywhere.
Green	#00775f	Safe and resolved; P4.
Amber / orange	#965800 · #b43b0a	Warnings; P3 and P2.

Typography is Manrope for headings, Inter for text and JetBrains Mono for codes and numbers. All tokens live in `src/styles.css` and are described in `DESIGN.md`.

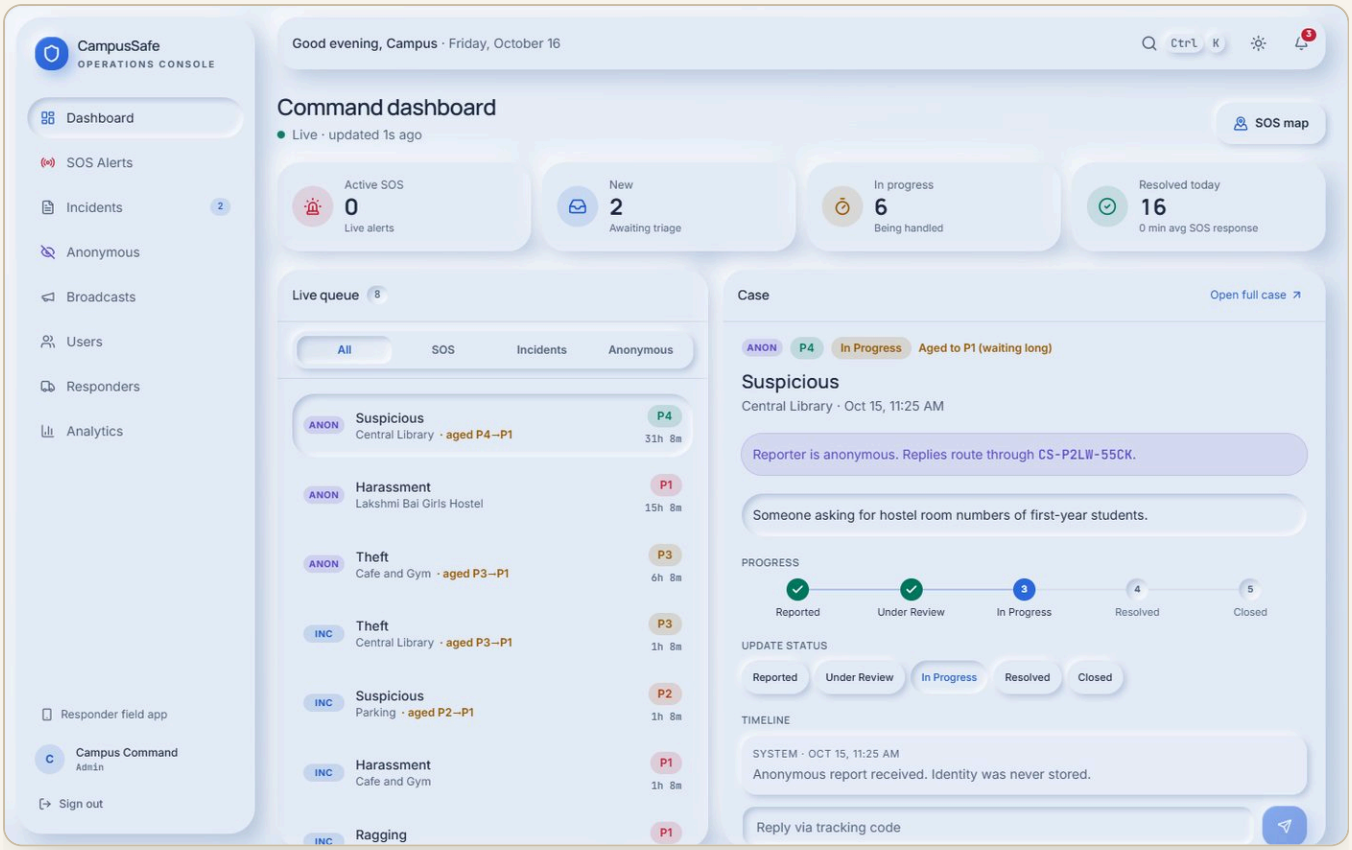


FIG 5 Command dashboard: live counters, the aged priority queue ("aged P4→P1"), and the selected anonymous case with its tracking code, 5-step status and timeline.

Built for stress, on a phone

- Large touch targets, high contrast in both themes, reduced motion respected; meaning is never carried by colour alone. The target is **WCAG 2.2 AA**.
- Phone layout has a bottom tab bar with the SOS button in the centre; every screen was checked at 360 px and 390 px widths.
- Installable (web app manifest, `display: standalone`), so it opens like an app from the home screen.
- Admins get a **Ctrl/Cmd+K** command palette, a notification bell and shimmering loading placeholders instead of spinners.

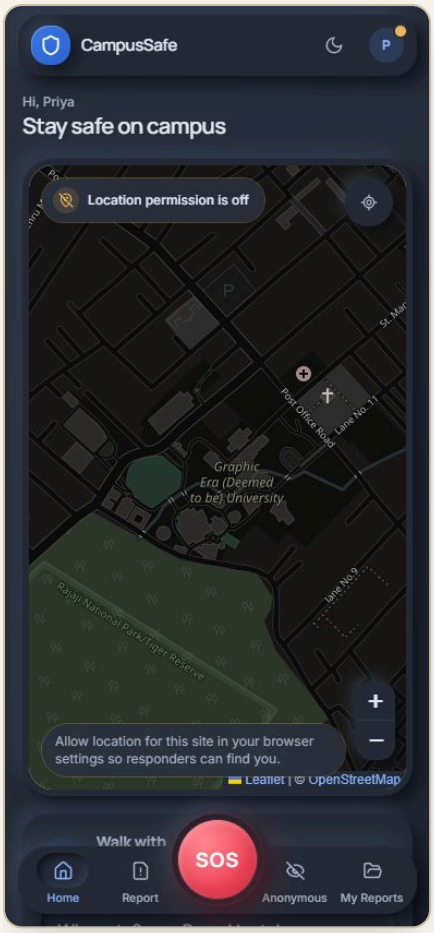


FIG 6 Student home on a phone, dark theme, with the SOS button in the tab bar.

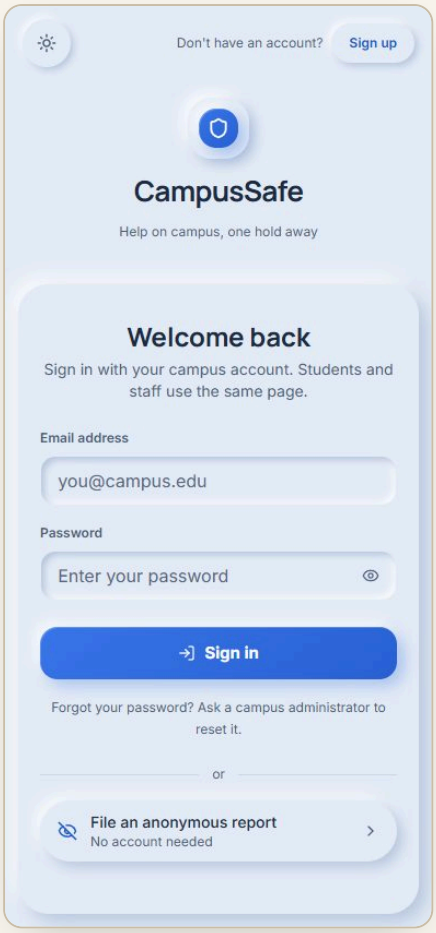


FIG 7 Sign-in on a phone, light theme.

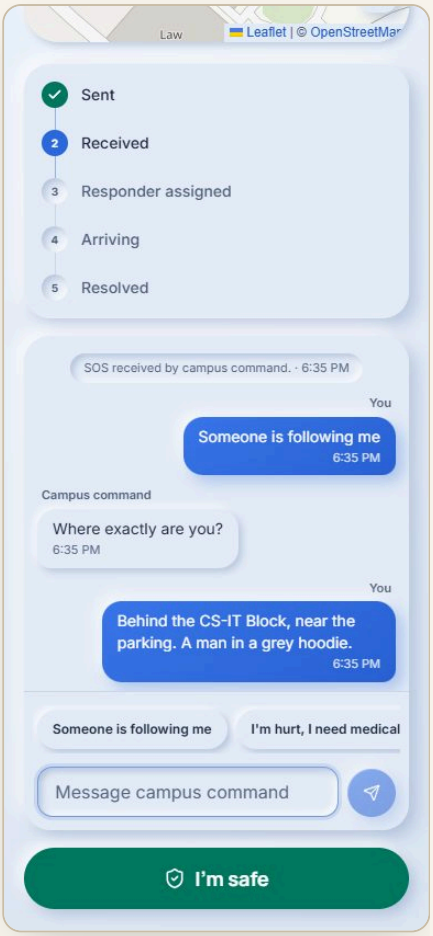


FIG 8 SOS chat with quick replies.

10 TRUST

Security, privacy and abuse protection

Accounts and sessions

- **Better Auth** email-and-password sign-in, minimum 8 characters, with a password-strength meter. Passwords are hashed by Better Auth and never stored in plain text.
- Students can only sign up as `student`. `SIGNUP_ALLOWED_DOMAINS` can restrict sign-up to campus email domains. Responders and admins are created by an admin; the first admin comes from `BOOTSTRAP_ADMIN_EMAIL` / `_PASSWORD`.
- Session cookies use the `__Host-` prefix, which forces Secure, `Path=/` and no Domain, so no other site or subdomain can set a cookie with the same name.
- A signed session cookie is cached for 5 minutes so most page loads skip a database round trip.
- Credentialed auth requests are checked against a list of trusted origins (CSRF protection).

Authorization on the server, not in the UI

Hiding a button is never the protection. Every server function checks the caller's role and ownership itself (`requireAdmin`, "is this your SOS?"). Folder guards (`admin/route.tsx`, `_student/route.tsx`) keep people out of pages they can't use, and `lib/roles.ts`, the map of which role may open which page, has 12 unit tests.

Input and data

- Every input passes a `zod` schema: coordinates in range, text lengths, enum values.
- Every query is parameterised through tagged SQL templates, so user input can never become SQL.
- Photos must be PNG, JPEG or WebP data and under about 3 MB; lists never return photos.
- Important actions are written to `audit_log` with the actor and subject.

Rate limits

A sliding-window limiter (`core/rate-limit.ts`) protects the endpoints most open to abuse:

ACTION	LIMIT	KEYED BY
File an incident	10 per hour	user
Create an anonymous report	5 per hour	salted IP hash
Look up a tracking code	30 per 10 min	salted IP hash (stops code guessing)
Reply on an anonymous report	20 per hour	salted IP hash
Reply in SOS chat	30 per 10 min	user

ANONYMITY, IN DETAIL

An anonymous report stores no user id, email or IP. The rate-limit key is a SHA-256 of the IP plus a random salt created when the server process starts and kept only in memory, so keys cannot be reversed or linked across restarts. The tracking code is the only credential, which is why it uses cryptographic randomness and why lookups are rate-limited.

Location privacy

Location is used to send an SOS or run a Safe Walk. Browsers ask the student for permission and only allow GPS over HTTPS. When permission is off, the app says so plainly on the map ("Location off · allow it in browser settings") rather than failing quietly.

QUALITY

11 Testing and quality

Automated tests: 43, run with `npm test`

Tests use Node's built-in test runner (`node --test`), with TypeScript run directly through `--experimental-strip-types`. They target the logic where a mistake is most costly and easiest to test without a browser.

FILE	TESTS	WHAT IT PROVES
`src/lib/roles.test.ts`	12	Each role can open exactly the pages it should, and no others
`src/lib/scheduling.test.ts`	10	Aging, queue order, semaphore limits and responder selection
`scripts/migration-plan.test.mjs`	7	Pending migrations come back in name order, applied ones never re-run, and the auth schema copy matches its source
`scripts/sign-out-plan.test.mjs`	6	Sign-out redirects only after the server confirms, and a hung or failed sign-out times out safely
`scripts/run.test.mjs`	4	The child-process runner forwards signals and exit codes
`scripts/write-atomic.test.mjs`	4	Atomic file writes never leave a half-written file

Static checks

- **TypeScript** in strict mode (`npm run typecheck`), with types shared between page and server.
- **ESLint** with React Hooks rules (`npm run lint`) and **Prettier** formatting.

Visual review

Every screen was captured with Playwright in **light and dark**, on **desktop and phones (360 and 390 px)**, for student, responder, admin and public pages, then compared before and after each redesign. Those captures found real bugs: sideways overflow on the student home, a clipped SOS counter and an oversized map on phone admin screens, and shadows clipped by the side column.

> **GAPS, STATED PLAINLY**
>
> There are no automated end-to-end tests of the SOS flow, no tests that run against a database, and no CI workflow in the repository; tests and checks are run by hand before pushing. Chapter 15 puts these first.

OPERATIONS

12 Deployment and operations

From git push to live

1. Code is pushed to GitHub (`github.com/1anc6/Campus-Safe`).
2. Vercel builds it: `vite build` produces the Nitro server output in `.vercel/output`.
3. The same build then runs `npm run db:migrate`, which applies any new SQL files to Neon, one at a time, each in a transaction, and records them in `_migrations`.
4. Vercel deploys the new version. Every later push redeploys.

Environment variables

VARIABLE	NEEDED FOR	PURPOSE
`DATABASE_URL`	production	Neon Postgres connection string (the pooled one)
`BETTER_AUTH_SECRET`	production	Long random string that signs sessions
`BETTER_AUTH_URL`	production	Public site URL
`BOOTSTRAP_ADMIN_EMAIL / _PASSWORD`	first run	Creates the first admin account
`SIGNUP_ALLOWED_DOMAINS`	optional	Comma-separated email domains allowed to sign up
`SEED_SAMPLE_DATA`	optional	`true` loads demo incidents into a real database
`CRON_SECRET`	optional	Enables the Safe Walk watchdog endpoint

Running it locally

`npm ci` then `npm run dev:local` starts the app on port 8080 with test accounts (an admin and a student) from `.env.local.example`, using in-memory PGLite. To use a real Neon database locally, put `DATABASE_URL` in a git-ignored `.env.local` and run the migration script first.

Operational notes

- **Migrations are append-only.** A new change is always a new `000X_name.sql` file; foreign keys are added `NOT VALID` so old data can never block a deploy.
- **GPS needs HTTPS**, so phones must be tested on the deployed site.
- **Map tiles** are free OpenStreetMap tiles; heavy production traffic should move to a tile provider with an API key.
- **Safe Walk escalation while the app is closed** depends on an external scheduler calling the cron endpoint every minute.

Where to change common things

I WANT TO…	EDIT
Change a page's layout	`src/routes/<page>.tsx`
Change the student tab bar or SOS button	`components/student/student-shell.tsx`, `components/shared/sos-button.tsx`

I WANT TO…	EDIT
Add a backend action	A new `createServerFn` in `src/backend/functions/`
Add a column or table	A new `migrations/000X_name.sql` (never edit old ones)
Change campus places or the security phone number	`src/lib/campus.ts`
Change colours or dark mode	`src/styles.css`

13 HONEST ASSESSMENT

Limitations

Read this chapter before relying on CampusSafe in a real emergency or making claims about it.

Adoption and real-world readiness

- **Not formally adopted by the university.** It is a working system built for Graphic Era, not an official service.
- **Responder phone numbers are placeholders** and are labelled as such in the app; they must be replaced with real ones.
- **No measured performance.** There are no response-time, load or uptime figures yet, so none should be claimed.
- **Not a replacement for emergency services.** It routes to campus command, not to police or ambulance.

Real-time and alerting

- Consoles **poll** every few seconds; there are no WebSockets. Fine at campus scale, but not instant.
- **No push notifications.** A responder or admin who has closed the app does not get alerted on their phone.
- Location depends on the phone's GPS: it can be poor indoors, and it needs HTTPS and the student's permission.

Scaling

- The rate limiter and the 3-second counter cache are **in memory, per server instance**. On serverless hosting with many instances the limits are weaker; a shared store such as Redis/Upstash is needed.
- Photos are stored as base64 text in Postgres rows. That is simple, but it grows the database fast; object storage would be better.
- Free OpenStreetMap tiles are not meant for heavy traffic.

Engineering

- No end-to-end tests of the SOS path, no database tests and no CI workflow (see chapter 11).
- Local development data resets on restart (PGLite in memory), by design.
- English only.

14 REFERENCE

Complete inventory of everything used

Languages and formats

TypeScript (60 `.tsx` + 39 `.ts` files) · JavaScript ES modules (10 `.mjs` scripts) · SQL for PostgreSQL (10 files) · CSS (Tailwind) · JSON · Markdown.

Frontend

LIBRARY	USED FOR
React 19, React DOM	UI
TanStack Router, router plugin	File-based pages and route guards
TanStack Query, TanStack Table	Data fetching and caching; admin tables
Tailwind CSS 4, tw-animate-css	Styling and animation
Radix UI primitives (dialog, select, popover, tabs, tooltip…)	Accessible building blocks for soft-UI components
Leaflet	Interactive campus maps over OpenStreetMap
Recharts	Analytics charts
cmdk	Ctrl/Cmd+K command palette
react-hook-form, @hookform/resolvers	Forms with schema validation
sonner, vaul	Toasts; mobile drawers
lucide-react	Icons
date-fns, react-day-picker	Dates and date picking
zustand	Small client-side state
class-variance-authority, clsx, tailwind-merge	Component variants and class names

Backend and data

LIBRARY OR SERVICE	USED FOR
TanStack Start	Server rendering and server functions on Node.js
Nitro	Server build output for Vercel
Better Auth	Email/password sign-in and sessions
Kysely, pg	SQL access to Postgres
@electric-sql/pglite	In-memory Postgres for local development
zod 4	Input validation

LIBRARY OR SERVICE	USED FOR
jose	Signed tokens
Neon	Serverless PostgreSQL in production
Vercel	Hosting, builds and deploys from GitHub
OpenStreetMap	Map tiles and building coordinates (ODbL)

Tooling

TOOL	USED FOR
Vite 8, @vitejs/plugin-react, lightningcss	Dev server and production build
TypeScript 5	Type checking
ESLint 9, typescript-eslint, React Hooks plugin	Linting
Prettier 3	Formatting
Node test runner	Unit tests
Playwright	Screenshots of every screen for visual review
Git and GitHub	Version control and pull-request review with teammates

15 ROADMAP

Future work

Ordered by how much each would improve safety, not by effort.

#	ITEM	WHY
1	Real responder numbers and university sign-off	Turns a working system into a service people can depend on.
2	Web Push notifications	Alert responders and admins even when the app is closed.
3	End-to-end tests of the SOS path, and CI	Prove on every push that hold → send → assign → resolve still works.
4	Shared rate-limit and cache store (Redis/Upstash)	Keep limits correct across many serverless instances.
5	Live responder location	Show the student how far away help is.
6	SMS fallback for SOS	Reach command when mobile data fails.
7	Object storage for photos	Keep the database small and fast.
8	Server-sent events or WebSockets	Replace polling with instant updates on the console.
9	Hindi and other languages	Help students who are more at ease outside English, especially under stress.
10	Formal WCAG 2.2 AA audit	Verify the accessibility target with assistive technology.

16 FOR STUDY

Glossary and credits

Credits

Author

Lanchenba Mutum: product design, architecture, frontend, backend, database, scheduling logic and documentation.

Contributors

Monica Agum and **Lydia1616**: pull requests #1–#11 covering the README files, `PRODUCT.md`, `package.json`, Prettier config, styles, profile code and build scripts.

Map data © OpenStreetMap contributors, available under the Open Database Licence (ODbL). Built for Graphic Era (Deemed to be) University, Clement Town, Dehradun; no endorsement by the university is implied.

Glossary

SOS
An emergency alert raised by holding the SOS button; carries live GPS.

Server function
A backend function a page calls like a normal function; the framework turns it into an HTTPS request.

`__Host-` cookie
A cookie the browser only accepts if Secure, `Path=/` and without a Domain.

Haversine
Formula for distance between two points on a sphere from their latitudes and longitudes.

Priority scheduling
Serving the most important job first.

Counting semaphore
A counter that limits how many holders a resource has; P() takes a slot, V() returns it.

Race condition
A bug where the result depends on which of two simultaneous actions happens first.

Optimistic concurrency
Assume no conflict, then check before saving and undo if someone else changed it.

Exponential backoff
Retrying with growing gaps (4 s, 8 s, 15 s…) so a failing network is not flooded.

PWA
Progressive web app: a website that installs to the home screen and opens like an app.

Session cookie
A small signed value the browser sends with each request to prove who is signed in.

CSRF
Cross-site request forgery: another site tricking your browser into acting for you; blocked by origin checks.

Geofence
A virtual boundary; here a 600 m circle around the campus centre.

Starvation / aging
A low-priority job waiting forever; aging raises its priority the longer it waits.

Round-robin
Taking turns; here, ties go to the unit that waited longest.

Compare-and-swap
Change a value only if it still has the expected old value; one racer wins.

Watchdog timer
A timer that triggers an action if something does not check in on time.

Sliding-window rate limit
At most N actions in any rolling time window.

Migration
A numbered SQL file that changes the schema; applied once, in order.

Partial index
An index over only the rows matching a condition, such as open cases.

View
A stored query that can be read like a table.

Trigger
Code the database runs automatically on an insert, update or delete.

`NOT VALID` foreign key
A key checked on new rows immediately, with old rows validated separately.

PGLite / Neon
Postgres in WebAssembly in memory; serverless cloud Postgres.

Wake Lock
A browser API that asks the device not to dim or sleep the screen.

Neumorphism
Soft UI: controls shaped by light and shadow on the same colour as the page.

— END OF DOCUMENT —